



1.- Descripción de la tarea.



Caso práctico

Ejercicio 1

Realizar una aplicación cuya descripción se compone de dos partes: la descripción de requisitos funcionales y no funcionales para establecer las bases del desarrollo de la lógica computacional solicitada e información relativa a que dicha solución contenga elementos que, más allá de un correcto funcionamiento, permitan entender la solución planteada.

- **Respecto a la lógica computacional:** tal y como hemos visto en los contenidos del tema, a través de las explicaciones y los ejemplo de código fuente de Java en el entorno Netbeans, la propuesta de tarea de este ejercicio aborda el clásico problema de la pareja "productor-consumidor". Un único hilo productor (clase **Fábrica**) abastece de elementos (números enteros) a una pareja de hangares de reparto (instancias de la clase **Hangar**). La mercancía (números enteros) debe ser retirada para su correspondiente reparto mediante dos hilos consumidores (clase **Camion**). Es importante señalar que cada camión trabaja con un único hangar (hay por tanto 2 instancias de hangar, cada una asociada a través del constructor de la clase Camion a cada uno de éstas). Respecto a la producción de elementos (números enteros) dispondremos, como ya se indicó antes, de una única instancia de la clase Fabrica la cual provee de números enteros calculados de forma aleatoria en el intervalo 0-499.
 - De cara a plantear la solución resulta esencial realizar las siguientes aclaraciones:
 - La ejecución del hilo fábrica almacena elementos en los hangares de forma balanceada. Esto quiere decir que deposita el número entero recién creado en el hangar que menos elementos contiene en ese momento. Al igual que en los ejemplos de la unidad de contenidos, en este caso, aplicaremos primitivas de tipo sleep asociadas a un coeficiente aleatorio y podría darse una situación en la cual, en este proceso de crear y depositar elementos en los hangares, una de las instancias de la clase Hangar tuviera más elementos (o menos) que la otra y, por ello, el hilo de la clase Camion vinculado a esa instancia será necesario que retire o procese más o menos unidades que el otro. Un ejemplo concreto de esto puede ser el siguiente: la fábrica proporciona 20 elementos (números enteros aleatorios comprendidos en el rango 0-499) y deposita 11 o 12 en el hangar asociado a la instancia del primer camión y, por este motivo, el hangar asociado a la instancia del segundo camión solo tendrá 9 u 8 elementos (aunque igualmente podría



haber sucedido que ambos hangares hubiesen tenido el mismo número de elementos).

- Cada hangar tiene la misma capacidad máxima. Siguiendo los valores del ejemplo anterior, si decidimos que la fábrica produzca 20 elementos, nunca se podrán almacenar, en un mismo instante de tiempo, más de 10 elementos en cada hangar (cada hangar tendrá como valor máximo la mitad entera del número de elementos que produzca la fábrica). Pero sí sería posible que la aleatoriedad asociada al almacenamiento de elementos por parte del hilo de la fábrica así como las aleatoriedades asociadas a la retirada de elementos de los hangares de los hilos de los camiones provoque que, al haber finalizado la ejecución, un hilo camión hubiera retirado más elementos totales de su hangar que el otro (y, por ese motivo, la fábrica hubiera depositado más elementos en ese hangar).
- Cada hilo asociado a un camión tiene un tiempo de espera (con un rango de aleatoriedad) similar, entendiéndose similar como valores dentro de un mismo rango (por ejemplo entre 0 y 99 milisegundos). Esta ausencia de determinismo provoca que el resultado final de cada ejecución del programa solicitado sea ligeramente diferente. Por ejemplo unas veces el camión 1 procesa 9 elementos y el camión 2 procesa 11 elementos, otras veces 10 y 10 elementos respectivamente, otras veces 11 y 9 elementos respectivamente, otras veces 12 y 8 elementos respectivamente, etc.
- Para evitar "interbloqueos" la operación de recogida de elementos del hangar por parte de un camión tendrá un tiempo de espera máximo definido. Esto se puede organizar de forma sencilla: la primitiva wait tiene un argumento que es el tiempo de espera máximo determinado (vamos a establecer, por ejemplo, 5 veces más que el tiempo máximo de espera entre dos agregaciones consecutivas de elementos por parte del hilo de la fábrica). Lo anterior, si bien flexibiliza y mejora la probabilidad de que se recoja elementos del hangar, no lo garantiza. Valore, para esas situaciones, emplear algún control de intentos (por ejemplo un máximo de 5 intentos tras lo cual se desiste y finaliza la ejecución).

■ **Respecto a la legibilidad de la solución planteada:** es fundamental agregar sentencias de texto que informen, en la salida estándar, la evolución de la ejecución. Deberán ser establecidas en partes críticas o fundamentales del código fuente para entender, y demostrar, que el funcionamiento es correcto y responde fielmente a la naturaleza del enunciado que se acaba de plantear en la tarea anterior. Para mejorar la comprensión de este ejercicio se proporcionan algunos extractos de los mensajes escritos en la salida estándar: [extracto 1](#) (PSP02_Tarea_Ejercicio1_salidaPorConsola1.txt), [extracto 2](#) (PSP02_Tarea_Ejercicio1_salidaPorConsola2.txt), [extracto 3](#) (PSP02_Tarea_Ejercicio1_salidaPorConsola3.txt) y [extracto 4](#) (PSP02_Tarea_Ejercicio1_salidaPorConsola4.txt).

Ejercicio 2

Realizar una aplicación cuya descripción se compone de dos partes: la descripción de requisitos funcionales y no funcionales para establecer las bases del desarrollo de la lógica computacional solicitada e información relativa a que dicha solución contenga

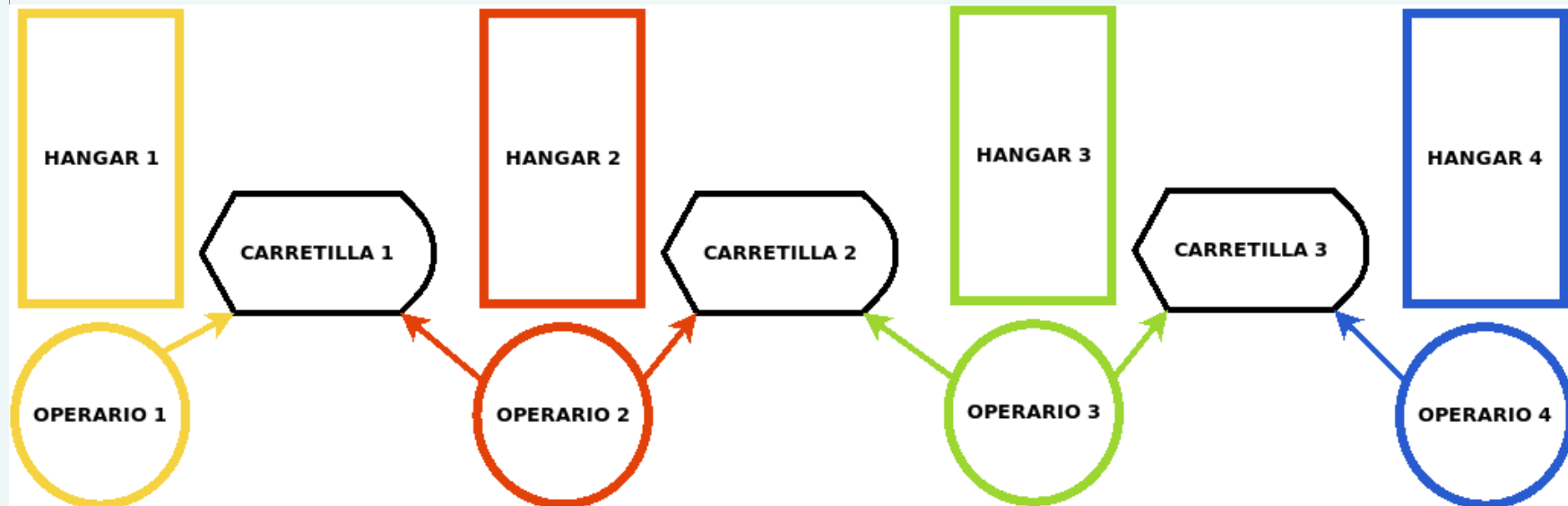


elementos que, más allá de un correcto funcionamiento, permitan entender la solución planteada.

- **Respecto a la lógica computacional:** tal y como hemos visto en los contenidos del tema, a través de las explicaciones y los ejemplo de código fuente de Java en el entorno Netbeans, la propuesta de tarea de este ejercicio aborda el desarrollo de una solución para arbitrar el uso de recursos compartidos entre hilos teniendo en cuenta que tenemos menos recursos que hilos susceptibles de necesitarlos y, por ello, deberá enfocar la solución haciendo uso de semáforos (clase Semaphore). La descripción del problema es como sigue a continuación:
 - En un hangar de carga tenemos 4 muelles de carga y, por cada uno de éstos, un operario que maneja una carretilla con la cual, cuando llega un camión, utiliza para realizar la carga/descarga de mercancía.
 - Los 4 muelles de carga están dispuestos linealmente de forma que, empezando de izquierda a derecha (vistos en perspectiva cenital) tenemos:
 - muelle de carga/descarga 1 y el operario 1
 - muelle de carga/descarga 2 y el operario 2
 - muelle de carga/descarga 3 y el operario 3
 - muelle de carga/descarga 4 y el operario 4
 - Tras un estudio estadístico de uso se ha determinado que, la mayor parte del tiempo, no es necesario que cada operario tenga de forma exclusiva una carretilla para las operaciones de carga/descarga ya que no es habitual que, a la misma vez, sea necesario utilizar todas estas máquinas. Es por ello que en un espacio adecuado y suficiente entre 2 hangares consecutivos se ubica una carretilla de tal forma que:
 - entre el hangar 1 y el hangar 2 se encuentra la carretilla número 1 (utilizable por los operarios de dichos hangares que son operario 1 y operario 2 respectivamente)
 - entre el hangar 2 y el hangar 3 se encuentra la carretilla número 2 (utilizable por los operarios de dichos hangares que son operario 2 y operario 3 respectivamente)
 - entre el hangar 3 y el hangar 4 se encuentra la carretilla número 3 (utilizable por los operarios de dichos hangares que son operario 3 y operario 4 respectivamente)
 - Como se habrá podido valorar hay operarios que solo pueden aspirar a utilizar una carretilla:
 - operario 1 solo podrá utilizar la carretilla que se ubica adyacente a él, esto es carretilla
 - operario 4 por idéntico motivo a operario 1, solo podrá utilizar carretilla 4



- Hay otros operarios que disponen de opción a solicitar una carretilla entre dos posibles opciones:
 - operario 2 podría utilizar la carretilla que se ubica de forma adyacente a su izquierda, esto es carretilla 1, o a su derecha, carretilla 2
 - operario 3 podría utilizar la carretilla que se ubica de forma adyacente a su izquierda, esto es carretilla 2, o a su derecha, carretilla 3
- No todo el tiempo los operarios necesitan utilizar las carretillas y, cuando les resulta necesario, lo hacen durante un tiempo finito.
- Para simular los tiempo de inactividad o de manipulación de las carretillas deberá establecer un margen aleatorio comprendido entre 0 y 0,5 segundos y 0 y 0,3 segundos respectivamente. Recuerde que para aplicar un cálculo de aleatoriedad comprendido dentro de un rango puede utilizar el método estático random de la clase Math.
- Como un ejemplo que ilustre la anterior descripción del contexto para el cual debe dar solución se indica ahora un pequeño esquema visual que será de ayuda:



- **Respecto a la legibilidad de la solución planteada:** es fundamental agregar sentencias de texto que informen, en la salida estándar, la evolución de la ejecución. Deberán ser establecidas en partes críticas o fundamentales del código fuente para entender, y demostrar, que el funcionamiento es correcto y responde fielmente a la naturaleza del enunciado que se acaba de plantear en la tarea anterior. Para mejorar la comprensión de este ejercicio, y más concretamente esta tarea segunda, se proporciona una posible colección de mensajes escritos en la salida estándar: [extracto](#).



(PSP02_Tarea_Ejercicio2_salidaPorConsola.txt).